Trajectory Optimization and LQR-Based Terminal Stabilization for a Double Inverted Pendulum on a Cart

Author Name
Department of Mechanical and Aerospace Engineering
Advanced Robotics Course Project Report

***Abstract*—This paper presents a detailed version-2 write-up of the double inverted pendulum trajectory optimization project, extended from an open-loop swing-up planner to a hybrid controller that combines nonlinear trajectory optimization, trajectory-tracking linear quadratic regulation, and final equilibrium LQR stabilization. The manuscript is intentionally written as both a theory document and an implementation document. First, the cart–double-pendulum dynamics are derived in manipulator form and then converted into the six-state nonlinear model used in the code. Second, the finite-horizon swing-up problem is formulated from a calculus-of-variations and Pontryagin minimum principle perspective so that the roles of the Hamiltonian, costate dynamics, and stationarity condition are explicit. Third, the practical direct multiple-shooting transcription used in CasADi is described exactly, including the symbolic RK4 defects, warm-start reference, feedforward control guess, cost weights, solver settings, and path constraints. Fourth, the paper explains why the optimized open-loop plan by itself is not sufficient for reliable terminal balancing, and derives the local linear feedback laws used after planning. In particular, a continuous-time equilibrium LQR is obtained from local linearization and the algebraic Riccati equation, while a finite-horizon trajectory-tracking gain schedule is obtained from a backward Riccati recursion on time-varying linearizations along the optimized trajectory. The resulting controller can be interpreted as a practical TO + LQR architecture: nonlinear trajectory optimization performs the global swing-up task, the trajectory-tracking gains keep the execution close to the nominal plan, and a final equilibrium LQR maintains the unstable upright equilibrium after arrival. This report is written without page-limit compression and explains the notation, derivations, and implementation details at a level appropriate for an advanced robotics course in which optimal control derivations are expected to be shown explicitly.**

***Index Terms*—trajectory optimization, calculus of variations, Pontryagin minimum principle, Hamiltonian, HJB, LQR, double inverted pendulum, cart-pole, CasADi**

I. Introduction

The double inverted pendulum on a cart is a representative underactuated robotic system in which a single horizontal cart force must coordinate translational motion of the base and rotational motion of two unstable pendulum links. Unlike a local balancing problem near an equilibrium, the swing-up task is fundamentally global and strongly nonlinear. Starting from a hanging configuration, the controller must inject energy, exploit inertial coupling, and steer the system into a neighborhood of the upright equilibrium from which a local stabilizer can take over.

This distinction between global motion generation and local stabilization is the central motivation for the present work. A local linear controller alone is inadequate from the downward configuration because the linear approximation only reflects the dynamics near a selected equilibrium. Conversely, a pure open-loop trajectory optimizer may produce an excellent finite-horizon plan in the model and still fail to hold the unstable upright state once the nominal horizon ends or once small modeling and execution deviations accumulate. The practical solution adopted in this project is therefore hybrid: use nonlinear trajectory optimization to generate the swing-up motion, then use LQR-based feedback to track the plan and stabilize the terminal equilibrium.

The existing project already contained a nonlinear CasADi trajectory optimizer for the double inverted pendulum. The purpose of this version-2 report is to document the extended formulation in a way that is suitable for an advanced robotics course. In particular, this manuscript does not stop at the code-level nonlinear program. It also includes the optimal-control derivation language that is often expected in a graduate course: variational reasoning, Hamiltonian construction, costate dynamics, stationarity conditions, and the relationship between Pontryagin's minimum principle and Hamilton–Jacobi theory.

The contributions of this manuscript are as follows.

1) The nonlinear dynamics are derived and explained using robotics notation consistent with the implementation.
2) The finite-horizon swing-up problem is formulated continuously, and the corresponding Hamiltonian and costate equations are written explicitly.
3) The direct multiple-shooting discretization used in the code is described exactly, including the reference warm start, feedforward input guess, path weights, terminal weights, and solver configuration.
4) The local terminal stabilizer is derived from linearization and the continuous-time algebraic Riccati equation.
5) The trajectory-tracking feedback implemented in the current code is interpreted as a finite-horizon LQR-style backward Riccati recursion computed along the optimized state and control trajectory.
6) The paper clarifies what changed from the original trajectory-optimization-only implementation: the nonlinear trajectory optimizer itself was preserved, while the execution layer was extended with trajectory-tracking

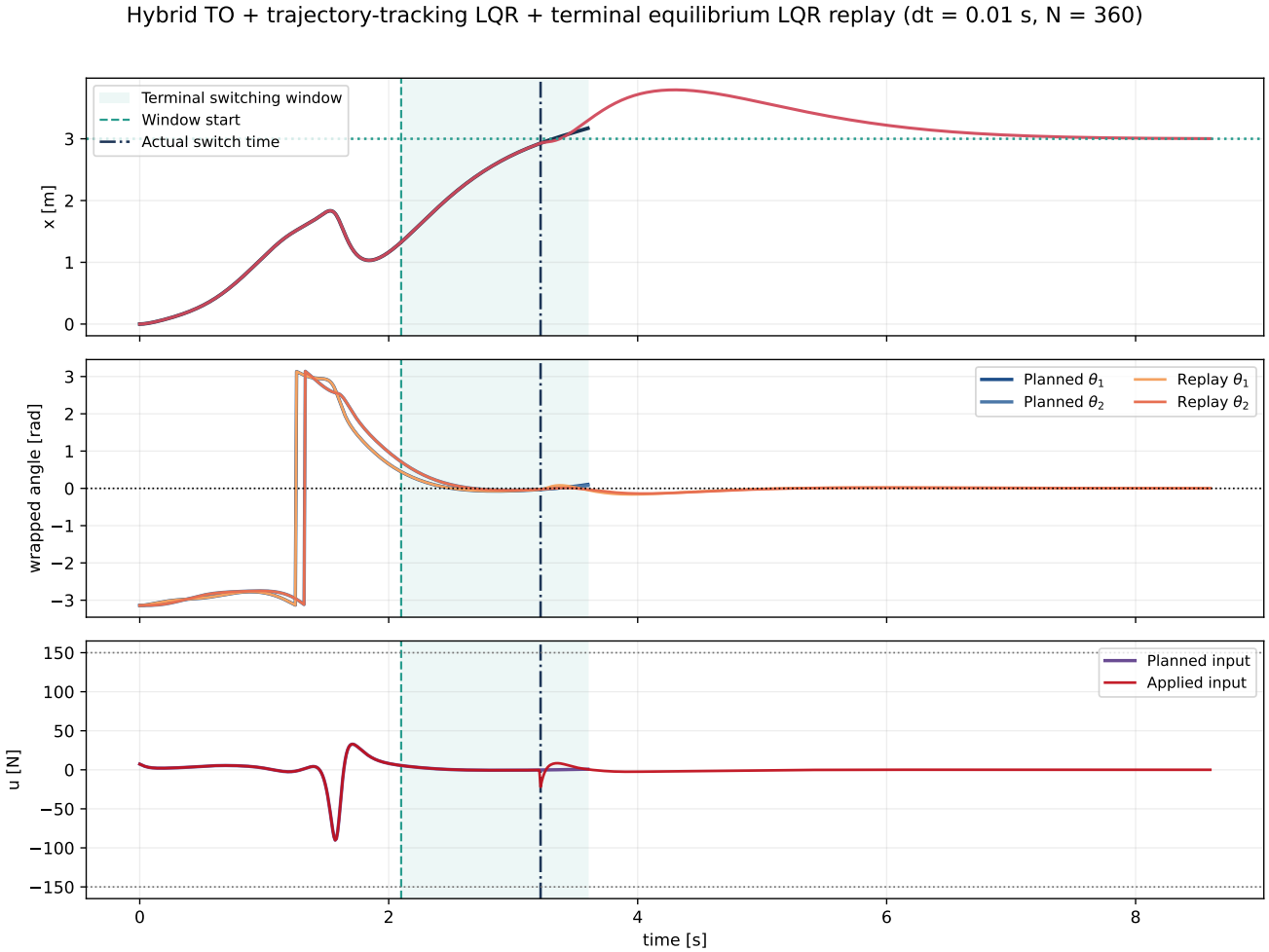

Fig. 1. Hybrid replay for the current controller architecture using the final figure-generation settings $N = 360$, $\Delta t = 0.01$ s, and $T_{\text{hold}} = 5.0$ s. The shaded region denotes the *terminal switching window*, not the exact interval in which the controller has already switched. The dashed vertical line marks the start of the window at $t = 2.1$ s, while the dash-dotted vertical line marks the actual switching instant at approximately $t = 3.22$ s. Thus the optimizer still plans through $t = 3.6$ s, but the controller is allowed to hand over earlier if the state has already entered the local basin of attraction of the final equilibrium LQR.

and final-equilibrium stabilization.

The rest of the paper proceeds from modeling, to continuous optimal control theory, to numerical transcription, and finally to hybrid control synthesis and implementation details.

II. System Model and Notation

A. Coordinates and State

Let the generalized coordinate vector be

$$\boldsymbol{q} = \begin{bmatrix} x & \theta_1 & \theta_2 \end{bmatrix}^\top, \tag{1}$$

where x is the horizontal cart position, θ_1 is the angle of the first pendulum link, and θ_2 is the angle of the second pendulum link. In the repository convention, both angles are measured from the upright direction. Hence

$$\theta_1 = \theta_2 = 0 \tag{2}$$

is the upright equilibrium, while the nominal hanging pose is near

$$\theta_1 = \theta_2 = \pi. \tag{3}$$

The full state vector is

$$\boldsymbol{x} = \begin{bmatrix} x & \theta_1 & \theta_2 & \dot{x} & \dot{\theta}_1 & \dot{\theta}_2 \end{bmatrix}^\top \in \mathbb{R}^6, \tag{4}$$

and the scalar control input is the horizontal cart force

$$u \in \mathbb{R}. \tag{5}$$

The default physical parameter structure in the model is

$$m_0 = 1.0 \text{ kg}, \quad m_1 = 0.2 \text{ kg}, \quad m_2 = 0.2 \text{ kg}, \tag{6}$$

$$l_1 = 0.5 \text{ m}, \quad l_2 = 0.5 \text{ m}, \quad g = 9.81 \text{ m/s}^2. \tag{7}$$

In the current TO + LQR demo script, these are combined with the modified damping and saturation values

$$d_x = 1.0, \quad d_1 = 0.1, \quad d_2 = 0.01, \quad u_{\max} = 150 \text{ N}. \tag{8}$$

B. Kinematics

The cart position in the plane is

$$\boldsymbol{p}_c = \begin{bmatrix} x \\ 0 \end{bmatrix}. \tag{9}$$

The first link tip position is

$$\boldsymbol{p}_1 = \boldsymbol{p}_c + \begin{bmatrix} l_1 \sin\theta_1 \\ l_1 \cos\theta_1 \end{bmatrix}, \tag{10}$$

and the second link tip position is

$$\boldsymbol{p}_2 = \boldsymbol{p}_1 + \begin{bmatrix} l_2 \sin\theta_2 \\ l_2 \cos\theta_2 \end{bmatrix}. \tag{11}$$

This global-angle convention is simple and matches the implementation. It also implies that angular errors should not be treated as ordinary Euclidean differences. Instead, angle errors must be wrapped into the principal interval $(-\pi, \pi]$, which is why the code uses the function

$$\mathrm{wrap}(\theta) = (\theta + \pi) \bmod 2\pi - \pi. \tag{12}$$

C. Manipulator-Form Dynamics

The implemented equations of motion are written in second-order manipulator form,

$$\boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} = \boldsymbol{\tau}(\boldsymbol{q}, \dot{\boldsymbol{q}}, u), \tag{13}$$

where the mass matrix is

$$\boldsymbol{M}(\boldsymbol{q}) = \begin{bmatrix} m_0 + m_1 + m_2 & (m_1 + m_2) l_1 \cos\theta_1 & m_2 l_2 \cos\theta_2 \\ (m_1 + m_2) l_1 \cos\theta_1 & (m_1 + m_2) l_1^2 & m_2 l_1 l_2 \cos(\theta_1 - \theta_2) \\ m_2 l_2 \cos\theta_2 & m_2 l_1 l_2 \cos(\theta_1 - \theta_2) & m_2 l_2^2 \end{bmatrix} \tag{14}$$

and the generalized-force vector is

$$\boldsymbol{\tau} = \begin{bmatrix} u - d_x \dot{x} + (m_1 + m_2) l_1 \sin\theta_1 \dot{\theta}_1^2 + m_2 l_2 \sin\theta_2 \dot{\theta}_2^2 \\ -d_1 \dot{\theta}_1 - m_2 l_1 l_2 \sin(\theta_1 - \theta_2) \dot{\theta}_2^2 + (m_1 + m_2) g l_1 \sin\theta_1 \\ -d_2 \dot{\theta}_2 + m_2 l_1 l_2 \sin(\theta_1 - \theta_2) \dot{\theta}_1^2 + m_2 g l_2 \sin\theta_2 \end{bmatrix}. \tag{15}$$

The physical interpretation is standard but important. The diagonal of $\boldsymbol{M}(\boldsymbol{q})$ represents inertia associated with each generalized coordinate. The off-diagonal terms encode dynamic coupling. Since the single actuator acts only through u in the cart coordinate, all pendulum motion must be induced indirectly through these coupling terms.

Solving (13) for $\ddot{\boldsymbol{q}}$ gives the first-order nonlinear state dynamics

$$\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, u) = \begin{bmatrix} \dot{x} & \dot{\theta}_1 & \dot{\theta}_2 & \ddot{x} & \ddot{\theta}_1 & \ddot{\theta}_2 \end{bmatrix}^\top. \tag{16}$$

The implementation clips the force before inserting it into the dynamics:

$$u_{\mathrm{sat}} = \mathrm{clip}(u, -u_{\max}, u_{\max}). \tag{17}$$

III. Continuous Optimal Control Formulation

A. Swing-Up as a Finite-Horizon Problem

The continuous-time swing-up problem may be written as follows. Given an initial state $\boldsymbol{x}(0) = \boldsymbol{x}_0$, find a control signal $u(\cdot)$ over $t \in [0, T]$ that minimizes

$$J[u] = \phi(\boldsymbol{x}(T)) + \int_0^T L(\boldsymbol{x}(t), u(t), t)\, dt \tag{18}$$

subject to the nonlinear dynamics

$$\dot{\boldsymbol{x}}(t) = \boldsymbol{f}(\boldsymbol{x}(t), u(t)). \tag{19}$$

Here L is the running cost and ϕ is the terminal cost. In words, the controller is trying to make the entire trajectory economical according to L, while also arriving in a desirable terminal state according to ϕ.

In the current project, the goal state is

$$\boldsymbol{x}^\star = \begin{bmatrix} 3 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}^\top, \tag{20}$$

meaning the cart should arrive at $x = 3$ m and both links should be upright with near-zero velocity.

B. Calculus-of-Variations Viewpoint

To expose the optimality conditions expected in an advanced robotics course, introduce the augmented functional

$$\bar{J} = \phi(\boldsymbol{x}(T)) + \int_0^T \left[L(\boldsymbol{x}, u, t) + \boldsymbol{\lambda}^\top(t) \big(\boldsymbol{f}(\boldsymbol{x}, u) - \dot{\boldsymbol{x}} \big) \right] dt, \tag{21}$$

where $\boldsymbol{\lambda}(t) \in \mathbb{R}^6$ is the costate. The role of the costate is to enforce the state dynamics as a variational constraint.

Define the Hamiltonian

$$H(\boldsymbol{x}, u, \boldsymbol{\lambda}, t) = L(\boldsymbol{x}, u, t) + \boldsymbol{\lambda}^\top \boldsymbol{f}(\boldsymbol{x}, u). \tag{22}$$

Then (21) becomes

$$\bar{J} = \phi(\boldsymbol{x}(T)) + \int_0^T \left[H(\boldsymbol{x}, u, \boldsymbol{\lambda}, t) - \boldsymbol{\lambda}^\top \dot{\boldsymbol{x}} \right] dt. \tag{23}$$

Now take the first variation. Varying the state, input, and costate gives

$$\delta\bar{J} = \nabla\phi(\boldsymbol{x}(T))^\top \delta\boldsymbol{x}(T) + \int_0^T \left[\frac{\partial H}{\partial \boldsymbol{x}}^\top \delta\boldsymbol{x} + \frac{\partial H}{\partial u} \delta u + \frac{\partial H}{\partial \boldsymbol{\lambda}}^\top \delta\boldsymbol{\lambda} - \delta\boldsymbol{\lambda}^\top \dot{\boldsymbol{x}} \right. \tag{24}$$

Integrating the last term by parts,

$$\int_0^T -\boldsymbol{\lambda}^\top \delta\dot{\boldsymbol{x}}\, dt = \left[-\boldsymbol{\lambda}^\top \delta\boldsymbol{x} \right]_0^T + \int_0^T \dot{\boldsymbol{\lambda}}^\top \delta\boldsymbol{x}\, dt. \tag{25}$$

Since the initial state is fixed, $\delta\boldsymbol{x}(0) = 0$. Therefore

$$\delta\bar{J} = \big(\nabla\phi(\boldsymbol{x}(T)) - \boldsymbol{\lambda}(T) \big)^\top \delta\boldsymbol{x}(T) + \int_0^T \left[\left(\frac{\partial H}{\partial \boldsymbol{x}} + \dot{\boldsymbol{\lambda}} \right)^\top \delta\boldsymbol{x} + \frac{\partial H}{\partial u} \delta u + \left(\frac{\partial H}{\partial \boldsymbol{\lambda}} - \dot{\boldsymbol{x}} \right)^\top \delta\boldsymbol{\lambda} \right] dt. \tag{26}$$

Because the variations are arbitrary, the stationarity of (26) yields the familiar necessary conditions:

$$\dot{\boldsymbol{x}} = \frac{\partial H}{\partial \boldsymbol{\lambda}} = \boldsymbol{f}(\boldsymbol{x}, u), \tag{27}$$

$$\dot{\boldsymbol{\lambda}} = -\frac{\partial H}{\partial \boldsymbol{x}}, \tag{28}$$

$$0 = \frac{\partial H}{\partial u}, \tag{29}$$

$$\boldsymbol{\lambda}(T) = \nabla\phi(\boldsymbol{x}(T)). \tag{30}$$

Equation (28) is the costate equation that the professor would expect to see in a variational derivation. Equation (29) is the pointwise optimality condition. Together, (27)–(30) define a two-point boundary-value problem: the state is known at the initial time, the costate is known at the terminal time, and the two must be solved jointly.

C. Pontryagin Minimum Principle Interpretation

The variational calculation above is the standard entry point to Pontryagin's minimum principle (PMP). PMP states that if $u^\star(t)$ is an optimal control and $\boldsymbol{x}^\star(t)$ is its corresponding state trajectory, then there exists a costate $\boldsymbol{\lambda}^\star(t)$ such that

$$\dot{\boldsymbol{x}}^\star(t) = \frac{\partial H}{\partial \boldsymbol{\lambda}}\Big|_{(\boldsymbol{x}^\star, u^\star, \boldsymbol{\lambda}^\star)}, \tag{31}$$

$$\dot{\boldsymbol{\lambda}}^\star(t) = -\frac{\partial H}{\partial \boldsymbol{x}}\Big|_{(\boldsymbol{x}^\star, u^\star, \boldsymbol{\lambda}^\star)}, \tag{32}$$

$$u^\star(t) \in \arg\min_{u \in \mathcal{U}} H(\boldsymbol{x}^\star(t), u, \boldsymbol{\lambda}^\star(t), t), \tag{33}$$

with the terminal condition

$$\boldsymbol{\lambda}^\star(T) = \nabla \phi(\boldsymbol{x}^\star(T)). \tag{34}$$

For this project, the admissible control set is the saturation interval

$$\mathcal{U} = [-u_{\max}, u_{\max}]. \tag{35}$$

Hence the minimizing input must also respect hard actuator bounds.

The importance of PMP in this report is not that the code directly solves the costate boundary-value problem. It does not. Instead, the significance is conceptual: the CasADi nonlinear program is a numerical transcription of precisely this constrained finite-horizon optimal control problem. The multiple-shooting variables, defect constraints, and finite-dimensional objective are the discrete counterparts of the continuous equations above.

D. Hamilton–Jacobi–Bellman Viewpoint

An equally important theoretical perspective is dynamic programming. Let

$$V(\boldsymbol{x}, t) = \min_{u(\cdot)} \left[\phi(\boldsymbol{x}(T)) + \int_t^T L(\boldsymbol{x}(\tau), u(\tau), \tau)\, d\tau \right] \tag{36}$$

be the value function. Then V satisfies the Hamilton–Jacobi–Bellman (HJB) equation

$$-\frac{\partial V}{\partial t}(\boldsymbol{x}, t) = \min_{u \in \mathcal{U}} \left[L(\boldsymbol{x}, u, t) + \nabla_{\boldsymbol{x}} V(\boldsymbol{x}, t)^\top \boldsymbol{f}(\boldsymbol{x}, u) \right], \tag{37}$$

with terminal boundary condition

$$V(\boldsymbol{x}, T) = \phi(\boldsymbol{x}). \tag{38}$$

Equation (37) is the dynamic-programming counterpart to PMP. The term inside the minimum is exactly the Hamiltonian with the identification

$$\boldsymbol{\lambda}(t) = \nabla_{\boldsymbol{x}} V(\boldsymbol{x}(t), t). \tag{39}$$

Thus PMP and HJB are not competing theories; they are complementary ways of expressing the same optimal-control structure.

The reason HJB is particularly relevant here is that LQR can be derived directly from it. Once the dynamics are linearized near an equilibrium and the cost is approximated quadratically, the value function becomes quadratic, and the HJB PDE reduces to a Riccati equation. That is precisely the bridge from nonlinear optimal control to the local LQR stabilizer used at the end of the trajectory.

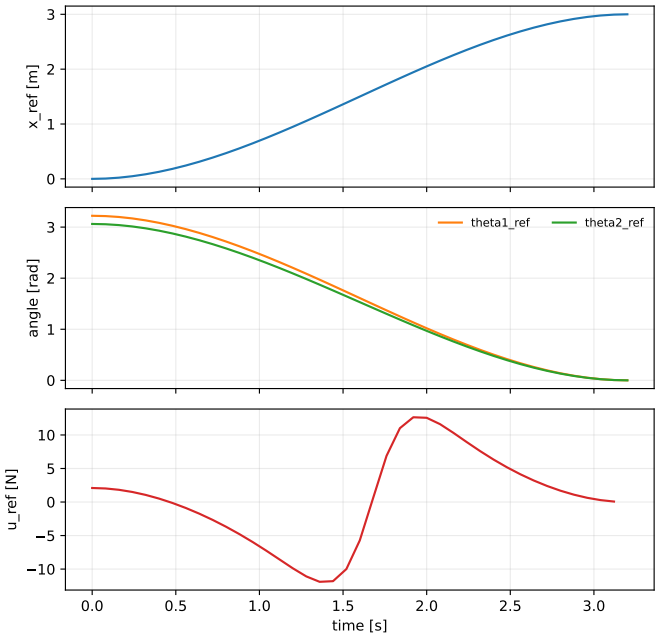

Fig. 2. Smooth swing-up reference and feedforward input initialization used to warm-start the CasADi trajectory optimizer. Even though this reference is not dynamically feasible, it gives the nonlinear program a physically meaningful starting point.

IV. Practical Nonlinear Trajectory Optimization Used in the Code

A. Reference Warm Start

The code warm-starts the nonlinear program using a smooth cubic interpolation between initial and goal states. Define the normalized horizon coordinate

$$\tau_k = \frac{k}{N}, \qquad k = 0, \ldots, N, \tag{40}$$

and the smoothstep profile

$$s(\tau) = 3\tau^2 - 2\tau^3. \tag{41}$$

Then the reference state is

$$x_k^{\mathrm{ref}} = x_0 + (x^\star - x_0) s(\tau_k), \tag{42}$$

$$\theta_{1,k}^{\mathrm{ref}} = \theta_{1,0} + (\theta_1^\star - \theta_{1,0}) s(\tau_k), \tag{43}$$

$$\theta_{2,k}^{\mathrm{ref}} = \theta_{2,0} + (\theta_2^\star - \theta_{2,0}) s(\tau_k), \tag{44}$$

while the reference velocities are set to zero.

This reference is not dynamically feasible. It is simply a good initial guess that keeps the optimizer away from pathological initializations.

B. Feedforward Input Initialization

For each reference state, a scalar feedforward input is computed by the bounded one-dimensional optimization

$$u_k^{\mathrm{ref}} \in \arg\min_{u \in [-u_{\max}, u_{\max}]} \left\| \ddot{\boldsymbol{q}}(\boldsymbol{x}_k^{\mathrm{ref}}, u) \right\|_2^2. \tag{45}$$

Intuitively, this chooses an input that makes the reference configuration as dynamically quiet as possible.

C. Decision Variables

The direct multiple-shooting transcription introduces the state matrix

$$\boldsymbol{X} = \begin{bmatrix} \boldsymbol{x}_0 & \boldsymbol{x}_1 & \cdots & \boldsymbol{x}_N \end{bmatrix} \in \mathbb{R}^{6\times(N+1)} \tag{46}$$

and the control matrix

$$\boldsymbol{U} = \begin{bmatrix} u_0 & u_1 & \cdots & u_{N-1} \end{bmatrix} \in \mathbb{R}^{1\times N}. \tag{47}$$

The initial node is constrained by

$$\boldsymbol{x}_0 = \boldsymbol{x}_{\text{init}}. \tag{48}$$

D. RK4 Defect Constraints

Dynamic feasibility is enforced with a symbolic RK4 step. For each $k = 0, \ldots, N-1$,

$$\boldsymbol{x}_{k+1} = \Phi_{\text{RK4}}(\boldsymbol{x}_k, u_k), \tag{49}$$

where

$$\boldsymbol{k}_1 = \boldsymbol{f}(\boldsymbol{x}_k, u_k), \tag{50}$$

$$\boldsymbol{k}_2 = \boldsymbol{f}\left(\boldsymbol{x}_k + \frac{\Delta t}{2}\boldsymbol{k}_1, u_k\right), \tag{51}$$

$$\boldsymbol{k}_3 = \boldsymbol{f}\left(\boldsymbol{x}_k + \frac{\Delta t}{2}\boldsymbol{k}_2, u_k\right), \tag{52}$$

$$\boldsymbol{k}_4 = \boldsymbol{f}(\boldsymbol{x}_k + \Delta t\boldsymbol{k}_3, u_k), \tag{53}$$

$$\Phi_{\text{RK4}}(\boldsymbol{x}_k, u_k) = \boldsymbol{x}_k + \frac{\Delta t}{6}(\boldsymbol{k}_1 + 2\boldsymbol{k}_2 + 2\boldsymbol{k}_3 + \boldsymbol{k}_4). \tag{54}$$

E. Stage and Terminal Cost Used in the Implementation

The repository uses the following quadratic and periodic cost components:

$$\boldsymbol{Q}_{\text{path}} = \text{diag}(8.0,\ 0,\ 0,\ 0.35,\ 0.2,\ 0.2), \tag{55}$$

$$\boldsymbol{Q}_{\text{term}} = \text{diag}(65.0,\ 0,\ 0,\ 5.0,\ 3.0,\ 3.0), \tag{56}$$

with angular penalties

$$w_{\theta,\text{path}} = 10.0, \tag{57}$$

$$w_{\theta,\text{term}} = 140.0, \tag{58}$$

and input regularization

$$w_u = 0.003, \tag{59}$$

$$w_{\Delta u} = 0.0008. \tag{60}$$

The angular mismatch is measured with the periodic function

$$\ell_{\text{ang}}(\theta, \theta_{\text{ref}}) = 1 - \cos(\theta - \theta_{\text{ref}}), \tag{61}$$

which avoids discontinuities from angle wrapping.

Thus the implemented finite-dimensional objective is

$$\begin{aligned} J_{\text{NLP}} = \sum_{k=0}^{N-1} \Big[& (\boldsymbol{x}_k - \boldsymbol{x}_k^{\text{ref}})^\top \boldsymbol{Q}_{\text{path}} (\boldsymbol{x}_k - \boldsymbol{x}_k^{\text{ref}}) \\ & + w_{\theta,\text{path}} \ell_{\text{ang}}(\theta_{1,k}, \theta_{1,k}^{\text{ref}}) + w_{\theta,\text{path}} \ell_{\text{ang}}(\theta_{2,k}, \theta_{2,k}^{\text{ref}}) \\ & + w_u (u_k - u_k^{\text{ref}})^2 + w_{\Delta u}(u_k - u_{k-1})^2 \Big] \\ & + (\boldsymbol{x}_N - \boldsymbol{x}^\star)^\top \boldsymbol{Q}_{\text{term}} (\boldsymbol{x}_N - \boldsymbol{x}^\star) \\ & + w_{\theta,\text{term}} \ell_{\text{ang}}(\theta_{1,N}, \theta_1^\star) + w_{\theta,\text{term}} \ell_{\text{ang}}(\theta_{2,N}, \theta_2^\star), \end{aligned} \tag{62}$$

where the smoothing term is omitted at $k = 0$.

TABLE I
FINAL PARAMETERS USED FOR THE VERSION-2 TO + LQR FIGURES

Item	Value
TO horizon steps N	360
TO timestep Δt	0.01 s
Planning horizon T	3.6 s
Post-plan hold time T_{hold}	5.0 s
Track bounds	$[-2.5,\ 7.5]$ m
Cart damping d_x	1.0
Joint damping (d_1, d_2)	$(0.1,\ 0.01)$
Force limit $u_{\max}$	150 N
$\boldsymbol{Q}_{\text{path}}$	$\text{diag}(8, 0, 0, 0.35, 0.2, 0.2)$
$\boldsymbol{Q}_{\text{term}}$	$\text{diag}(65, 0, 0, 5, 3, 3)$
$w_{\theta,\text{path}}$	10.0
$w_{\theta,\text{term}}$	140.0
w_u	0.003
$w_{\Delta u}$	0.0008
Tracking $\boldsymbol{Q}_{\text{track}}$	$\text{diag}(20, 140, 140, 8, 18, 18)$
Tracking $\boldsymbol{R}_{\text{track}}$	0.8
Equilibrium LQR $\boldsymbol{Q}$	$\text{diag}(30, 220, 220, 18, 32, 32)$
Equilibrium LQR $\boldsymbol{R}$	0.6
Switch thresholds	(0.16 rad, 0.8, 0.25 m)
Terminal switch window	1.5 s

F. Constraints and Solver Configuration

The control bounds are

$$-u_{\max} \le u_k \le u_{\max}. \tag{63}$$

If enabled, cart position bounds are imposed as

$$x_{\min} \le x_k \le x_{\max}. \tag{64}$$

In the current demo script, the replay stage uses

$$(x_{\min}, x_{\max}) = (-2.5, 7.5), \tag{65}$$

while the TO solve may or may not enforce those bounds depending on the command-line option.

The CasADi frontend uses IPOPT when available, with

$$\texttt{max\_iter} = 2000, \tag{66}$$

$$\texttt{tol} = 10^{-4}, \tag{67}$$

$$\texttt{acceptable\_tol} = 10^{-3}, \tag{68}$$

$$\texttt{acceptable\_iter} = 8. \tag{69}$$

The final visualization and hybrid replay results in this report use

$$N = 360, \qquad \Delta t = 0.01 \text{ s}, \tag{70}$$

which yields a planning horizon of

$$T = N\Delta t = 3.6 \text{ s}. \tag{71}$$

Thus the horizon duration is unchanged from the earlier $N = 120$, $\Delta t = 0.03$ configuration, but the trajectory is sampled three times more densely in time, which produces visibly smoother plots and a finer direct multiple-shooting discretization.

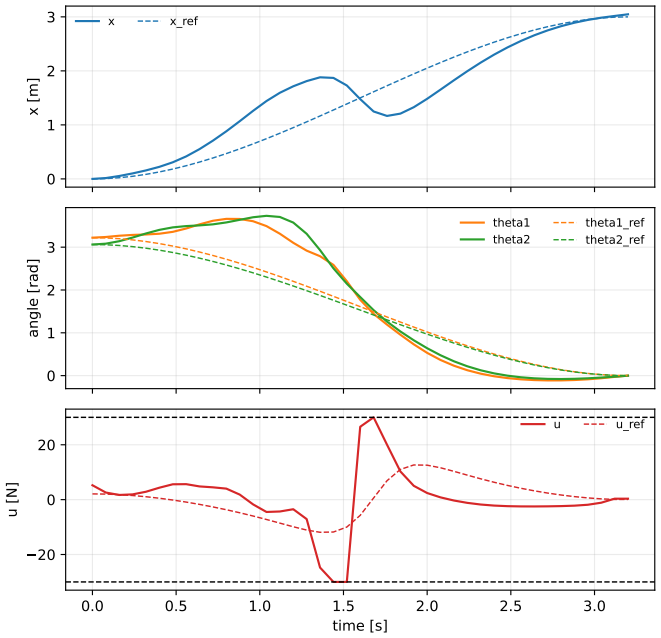

Fig. 3. Optimized swing-up plan produced by the nonlinear trajectory optimization module. This is the nominal motion that the version-2 controller tracks before switching to terminal stabilization.

V. Why Open-Loop Trajectory Optimization Alone Is Not Enough

Although the NLP in (62) can produce a feasible swing-up plan, the plan is still fundamentally finite horizon and model based. Two practical limitations follow immediately.

First, the optimizer solves a *terminal-reaching* problem, not an infinite-horizon stabilization problem. Even if the terminal state is near upright, the optimization itself does not guarantee that the system will remain there after the horizon ends.

Second, the optimized state sequence and input sequence are only nominal. During execution, even small mismatch in damping, numerical integration, or phase timing can move the nonlinear system away from the nominal path. For an unstable terminal equilibrium, these deviations can grow rapidly.

This is exactly why a TO + LQR architecture is natural. The trajectory optimizer supplies a globally meaningful nominal motion, and the LQR laws supply local feedback authority. In other words,

$$\text{TO plans how to arrive,} \qquad \text{LQR ensures how to stay.} \tag{72}$$

VI. Local Linearization and Final Equilibrium LQR

A. Linearization

Let $(\boldsymbol{x}_{\mathrm{eq}}, u_{\mathrm{eq}})$ denote the upright equilibrium pair. In the current task,

$$\boldsymbol{x}_{\mathrm{eq}} = \begin{bmatrix} 3 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}^{\top}, \qquad u_{\mathrm{eq}} = 0. \tag{73}$$

Linearizing the nonlinear system around this operating point yields

$$\delta\dot{\boldsymbol{x}} = \boldsymbol{A}\,\delta\boldsymbol{x} + \boldsymbol{B}\,\delta u, \tag{74}$$

where

$$\boldsymbol{A} = \left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\right|_{(\boldsymbol{x}_{\mathrm{eq}}, u_{\mathrm{eq}})}, \tag{75}$$

$$\boldsymbol{B} = \left.\frac{\partial \boldsymbol{f}}{\partial u}\right|_{(\boldsymbol{x}_{\mathrm{eq}}, u_{\mathrm{eq}})}. \tag{76}$$

The implementation computes these Jacobians numerically with centered finite differences.

B. LQR Cost and HJB Derivation

For the linearized system (74), define the infinite-horizon quadratic cost

$$J_{\mathrm{LQR}} = \int_0^{\infty} \left(\delta\boldsymbol{x}^{\top}\boldsymbol{Q}\delta\boldsymbol{x} + \delta u^{\top}\boldsymbol{R}\delta u\right) dt, \tag{77}$$

with

$$\boldsymbol{Q} = \mathrm{diag}(30,\ 220,\ 220,\ 18,\ 32,\ 32), \tag{78}$$

$$\boldsymbol{R} = [0.6]. \tag{79}$$

Now derive the feedback law by the HJB method. Assume the value function is quadratic:

$$V(\delta\boldsymbol{x}) = \delta\boldsymbol{x}^{\top}\boldsymbol{S}\delta\boldsymbol{x}, \tag{80}$$

where $\boldsymbol{S} = \boldsymbol{S}^{\top} \succcurlyeq 0$. Then

$$\nabla V(\delta\boldsymbol{x}) = 2\boldsymbol{S}\delta\boldsymbol{x}. \tag{81}$$

Substituting into the stationary HJB equation gives

$$0 = \min_{\delta u}\left[\delta\boldsymbol{x}^{\top}\boldsymbol{Q}\delta\boldsymbol{x} + \delta u^{\top}\boldsymbol{R}\delta u + 2\delta\boldsymbol{x}^{\top}\boldsymbol{S}(\boldsymbol{A}\delta\boldsymbol{x} + \boldsymbol{B}\delta u)\right]. \tag{82}$$

Differentiate with respect to δu:

$$2\boldsymbol{R}\delta u + 2\boldsymbol{B}^{\top}\boldsymbol{S}\delta\boldsymbol{x} = 0. \tag{83}$$

Therefore

$$\delta u = -\boldsymbol{K}\delta\boldsymbol{x}, \qquad \boldsymbol{K} = \boldsymbol{R}^{-1}\boldsymbol{B}^{\top}\boldsymbol{S}. \tag{84}$$

Substituting back into HJB yields the continuous-time algebraic Riccati equation (CARE):

$$\boldsymbol{A}^{\top}\boldsymbol{S} + \boldsymbol{S}\boldsymbol{A} - \boldsymbol{S}\boldsymbol{B}\boldsymbol{R}^{-1}\boldsymbol{B}^{\top}\boldsymbol{S} + \boldsymbol{Q} = 0. \tag{85}$$

This is exactly the LQR relation one would want to connect to Hamilton–Jacobi theory in a course report. The code solves (85) with `scipy.linalg.solve_continuous_are` and then computes the gain from (84). The implemented equilibrium control law is

$$u = u_{\mathrm{eq}} - \boldsymbol{K}\,\delta\boldsymbol{x}. \tag{86}$$

C. Direct Link Between the HJB Derivation and the Implemented LQR

Because the theoretical derivation can otherwise feel detached from the code, it is worth making the connection explicit. The final equilibrium LQR used in the implementation is precisely the controller obtained by applying the HJB derivation above to the linearized upright dynamics. In other words,

$$\text{nonlinear system } \boldsymbol{f}(\boldsymbol{x}, u) \xrightarrow{\text{linearize at } (\boldsymbol{x}_{\text{eq}}, u_{\text{eq}})} (\boldsymbol{A}, \boldsymbol{B}), \quad (87)$$

$$(\boldsymbol{A}, \boldsymbol{B}), (\boldsymbol{Q}, \boldsymbol{R}) \xrightarrow{\text{HJB with } V = \delta\boldsymbol{x}^\top \boldsymbol{S} \delta\boldsymbol{x}} \boldsymbol{S}, \quad (88)$$

$$\boldsymbol{S} \xrightarrow{\boldsymbol{K} = \boldsymbol{R}^{-1}\boldsymbol{B}^\top \boldsymbol{S}} \boldsymbol{K}_{\text{eq}}. \quad (89)$$

Therefore the matrix `s` stored in the code's `LQRGain` structure is the numerical Riccati solution $\boldsymbol{S}$, and the matrix `k` is exactly the gain $\boldsymbol{K}_{\text{eq}}$ predicted by the HJB derivation. The code is not merely "inspired by" HJB at this stage; it is implementing the Riccati solution that HJB produces for the local quadratic approximation.

VII. Trajectory-Tracking LQR Along the Optimized Swing-Up

A. Why a Single Final LQR Is Not Sufficient During the Entire Swing-Up

The final equilibrium LQR is valid only near the upright equilibrium. It is not appropriate for the entire swing-up because the system spends most of the maneuver far from the local linearization point. What is needed during the transient is a trajectory-tracking controller around the optimized nominal state and control sequence

$$\{\boldsymbol{x}_0^\star, \boldsymbol{x}_1^\star, \ldots, \boldsymbol{x}_N^\star\}, \qquad \{u_0^\star, u_1^\star, \ldots, u_N^\star\}. \quad (90)$$

B. Time-Varying Linearization Along the Plan

At each discrete stage k, linearize around the nominal pair $(\boldsymbol{x}_k^\star, u_k^\star)$:

$$\delta\dot{\boldsymbol{x}} = \boldsymbol{A}_k \delta\boldsymbol{x} + \boldsymbol{B}_k \delta u. \quad (91)$$

The implementation uses the finite-difference Jacobian routine already available in the repository, evaluated at each planned state and input.

With timestep $\Delta t_k = t_{k+1} - t_k$, the code forms a first-order discrete approximation

$$\boldsymbol{A}_{d,k} = \boldsymbol{I} + \Delta t_k \boldsymbol{A}_k, \quad (92)$$

$$\boldsymbol{B}_{d,k} = \Delta t_k \boldsymbol{B}_k. \quad (93)$$

C. Backward Riccati Recursion

The trajectory-tracking controller implemented in the current code uses the quadratic weights

$$\boldsymbol{Q}_{\text{track}} = \text{diag}(20,\ 140,\ 140,\ 8,\ 18,\ 18), \quad (94)$$

$$\boldsymbol{R}_{\text{track}} = [0.8]. \quad (95)$$

Starting from the final Riccati matrix

$$\boldsymbol{S}_N = \boldsymbol{S}_{\text{eq}}, \quad (96)$$

where $\boldsymbol{S}_{\text{eq}}$ is the CARE solution of the final equilibrium LQR, the code runs the backward recursion

$$\boldsymbol{G}_k = \boldsymbol{R}_{\text{track}} + \boldsymbol{B}_{d,k}^\top \boldsymbol{S}_{k+1} \boldsymbol{B}_{d,k}, \quad (97)$$

$$\boldsymbol{K}_k = \boldsymbol{G}_k^{-1} \boldsymbol{B}_{d,k}^\top \boldsymbol{S}_{k+1} \boldsymbol{A}_{d,k}, \quad (98)$$

$$\boldsymbol{S}_k = \boldsymbol{Q}_{\text{track}} + \boldsymbol{A}_{d,k}^\top \boldsymbol{S}_{k+1} (\boldsymbol{A}_{d,k} - \boldsymbol{B}_{d,k}\boldsymbol{K}_k). \quad (99)$$

This is the discrete finite-horizon LQR recursion. Conceptually, it is a local approximation to solving HJB backward in time along the nominal trajectory. Thus even though the implementation is practical and numerical, it still has a clean theoretical interpretation within the dynamic-programming framework.

For the final figures generated with $N = 360$ and $\Delta t = 0.01$ s, this recursion is carried out over 361 nominal states and 361 stored control samples, which yields a comparatively smooth scheduled gain sequence for trajectory tracking.

D. Implemented Tracking Control Law

Let the reference state at execution time t be obtained from linear interpolation of the planned states, and let $k(t)$ denote the associated index. The tracking error is

$$\delta\boldsymbol{x}_{\text{track}}(t) = \boldsymbol{x}(t) - \boldsymbol{x}^\star(t), \quad (100)$$

with the angular components wrapped into $(-\pi, \pi]$. The controller then applies

$$u(t) = u_{k(t)}^\star - \boldsymbol{K}_{k(t)}\, \delta\boldsymbol{x}_{\text{track}}(t). \quad (101)$$

Equation (101) is the key practical addition relative to the original open-loop TO implementation. The nonlinear optimizer itself was not replaced. What changed is the execution policy: instead of replaying the nominal input sequence blindly, the code now corrects deviations in real time using a linearized local feedback law.

VIII. Switch to Final Equilibrium LQR

The code switches from trajectory tracking to final equilibrium stabilization when either the planned horizon has ended or the state is sufficiently close to the upright equilibrium near the terminal portion of the plan. The current thresholds are

$$\max(|e_{\theta_1}|, |e_{\theta_2}|) \le 0.16 \text{ rad}, \quad (102)$$

$$\|\boldsymbol{e}_{\text{vel}}\|_2 \le 0.8, \quad (103)$$

$$|e_x| \le 0.25 \text{ m}, \quad (104)$$

inside a terminal window of

$$T_{\text{window}} = 1.5 \text{ s}. \quad (105)$$

Because the planning horizon is 3.6 s in the final setting, the switching window begins at

$$t_{\text{window,start}} = 3.6 - 1.5 = 2.1 \text{ s}. \quad (106)$$

This does *not* mean that the controller necessarily switches at 2.1 s. It only means that from 2.1 s onward, an early handoff is permitted if the state already satisfies the local position, angle,

and velocity conditions. In the final figure-generation run used in this report, the actual handoff occurred at approximately

$$t_{\text{switch}} \approx 3.22 \text{ s}, \tag{107}$$

which is later than the start of the switching window but still before the nominal plan end time 3.6 s.

After switching, the control law becomes

$$u(t) = -\boldsymbol{K}_{\text{eq}} \delta \boldsymbol{x}_{\text{eq}}(t), \tag{108}$$

where $\delta \boldsymbol{x}_{\text{eq}}$ is the state error relative to the final upright equilibrium.

The logic is simple but important. The trajectory-tracking law is designed to keep the system close to the finite-horizon nominal swing-up motion. Once the state has entered the basin where the final LQR is appropriate, it is preferable to use the infinite-horizon local stabilizer rather than continue following a finite-horizon transient plan.

IX. Final Hybrid Controller Architecture

Putting the pieces together, the implemented controller is

$$u(t) = \begin{cases} u^\star_{k(t)} - \boldsymbol{K}_{k(t)}(\boldsymbol{x}(t) - \boldsymbol{x}^\star(t)), & \text{during trajectory tracking,} \\ -\boldsymbol{K}_{\text{eq}}(\boldsymbol{x}(t) - \boldsymbol{x}_{\text{eq}}), & \text{after switching to terminal stabilization} \end{cases} \tag{109}$$

This can be summarized conceptually as

$$\boxed{\text{nonlinear TO for global swing-up} \;\rightarrow\; \text{time-varying local tracking feedback} \;\rightarrow\; \text{final equilibrium LQR}} \tag{110}$$

In the current demo script, the simulation duration is chosen as

$$T_{\text{display}} = T_{\text{plan}} + T_{\text{hold}}, \tag{111}$$

with default hold time

$$T_{\text{hold}} = 5.0 \text{ s}. \tag{112}$$

Thus if the TO horizon is 3.6 s, the animation continues for roughly 8.6 s so that the final LQR stabilization phase is visible instead of ending immediately at the arrival time.

In the final figure-generation run used in this manuscript, the optimized terminal state was

$$\boldsymbol{x}^\star_N \approx \begin{bmatrix} 3.1702 & 0.1045 & 0.0768 & 0.5904 & 0.5327 & 0.3140 \end{bmatrix}^\top, \tag{113}$$

the replayed hybrid response finished with wrapped error norm approximately 8.59×10^{-3}, and the peak replay input magnitude was approximately 90.15 N. These values are consistent with a trajectory that reaches the upright neighborhood with nonzero velocity and then settles under feedback rather than under open-loop terminal perfectness alone.

X. Discussion

A. What Stayed the Same and What Changed

It is important to be precise about the version-2 change set.

What stayed the same:

- the nonlinear model,
- the warm-start reference generation,
- the feedforward input guess,
- the direct multiple-shooting CasADi formulation,
- the path and terminal objective weights of the TO problem.

What changed:

- the optimized plan is no longer replayed purely open loop,
- a trajectory-tracking Riccati recursion is built along the optimized plan,
- a final equilibrium LQR is synthesized around the upright target,
- the execution policy switches from plan tracking to terminal balancing.

Therefore, this project should not be described as "replacing TO with LQR." It is more accurate to say that the original TO planner was retained and embedded inside a more complete feedback execution architecture.

B. Why the Variational Derivation Still Matters Even Though CasADi Solves an NLP

One might ask why it is necessary to show Hamiltonians and costates if the implementation ultimately uses direct multiple shooting in CasADi. The answer is pedagogical and conceptual. The numerical NLP did not appear from nowhere. It is a discrete realization of the same finite-horizon optimal control problem that one would write using calculus of variations or PMP. Showing the derivation makes the structure of the optimization problem explicit:

- the running cost in the Hamiltonian corresponds to the stage cost in the NLP,
- the dynamic constraint in the Hamiltonian corresponds to the RK4 defect constraint,
- the costate equation corresponds to backward sensitivity propagation,
- the local LQR laws emerge naturally from the HJB approximation near the trajectory or equilibrium.

For an advanced robotics course, these theoretical links are often as important as the numerical results themselves.

C. Limitations

Several limitations remain.

1) The TO problem is still sensitive to discretization and warm-start choice.
2) The trajectory-tracking gains are based on local linearization and first-order discretization, so they are only locally valid around the nominal path.
3) The replay often uses substantial control effort and may approach saturation.
4) The switching logic is heuristic rather than formally verified by invariant-set analysis.

These are typical tradeoffs in practical nonlinear control pipelines: the method is significantly more reliable than pure open-loop replay, but it is not a globally optimal feedback law.

XI. Conclusion

This paper presented a detailed version-2 formulation of a double inverted pendulum swing-up controller that combines nonlinear trajectory optimization with LQR-based feedback. The nonlinear trajectory optimization problem was first derived in the language of finite-horizon optimal control, including the Hamiltonian, costate equation, stationarity condition, and the connection to dynamic programming through the HJB equation. The practical CasADi implementation was then described exactly as used in the code: smooth reference warm start, feedforward input initialization, direct multiple shooting, RK4 defect constraints, bounded control, and weighted path and terminal penalties. Finally, the paper showed how local linear feedback is added in two layers: a time-varying trajectory-tracking Riccati recursion along the optimized swing-up, and a final equilibrium LQR derived from local linearization and the algebraic Riccati equation.

The main conceptual conclusion is straightforward but important. For the double inverted pendulum, trajectory optimization and LQR should not be viewed as competing approaches. They solve different parts of the same problem. Trajectory optimization handles the global nonlinear swing-up maneuver, while LQR provides the local feedback structure needed for reliable execution and terminal stabilization. That division of labor is both theoretically natural and practically effective, and it is precisely the architecture implemented in the current version of the project.